Lab 03: Object Oriented Design Principles (Class Level)

Introduction

Students have learned the theoretical concepts of class level object oriented design principles in lectures. In this lab, students will learn how to identify and fix pieces of codes where these principles are violated.

Objectives

After the completion of this lab, students will be able to identify which parts of software are violating class level object oriented principles and how to fix them.

Tools/Software Requirement

- Papyrus/Rational Rose

Description

It's a process of planning a software system where objects will interact with each other to solve specific problems. The saying goes, "Proper Object oriented design makes a developer's life easy, whereas bad design makes it a disaster."

Lab Task

Task 1

We have learned five principles of class level object oriented design formally known as SOLID principles. The five principles are:

- The Single Responsibility Principle (SRP)
- The Open Closed Principle (OCP)
- The Liskov Substitution Principle (LSP)

Your task:

- For each of the principles, you have to give two coding examples where the principles are violating.

- Given a brief explanation of why the piece of code is violating the principle.
- A refactored version of the code, where the principles are respected.
- Given a brief explanation of why the refactored version of the code is respecting the principle.

Important Note: You are not required to write a fully functional code. Only write enough code which can make your point.

Example

Liskov Substitution Principle

Violating Code:

```
class Rectangle{
   protected int width;
   protected int height;
   public void setWidth(int w){width = w;}
   public void setHeight(int h){height = h;}
   public int getWidth(){return width;}
   public int getHeight(){return height;}
   public int getArea(){return width * height;}
}

class Square extends Rectangle {
   public void setWidth(int  w){
      width = w;
      height = w;
   }

   public void setHeight(int h){
      width = h;
      height = h;
   }
}
```

Why the code is violating LSP

The class Square is extending the Rectangle class. Mathematically, there exist a strong relationship between a rectangle and square (square being a special form of rectangle where

width and height are equal). However, behaviorally a square object is completely different from a rectangle object. In the above code, the setWidth and setHeight overridden functions of Square class are changing the behavior of the same functions in the parent class. Thus the subclass is no more substitutable for the base class, which violates Liskov Substitution Principle.

Refactored Code

```
abstract class Shape{
   public void setWidth(int w){};
   public void setHeight(int h){};
}

class Rectangle extends Shape{
   protected int width;
   protected int height;
   public void setWidth(int w){width = w;}
   public void setHeight(int h){height = h;}
   public int getWidth(){return width;}
   public int getHeight(){return height;}
   public int getArea(){return width * height;}
}

class Square extends Shape {
   protected int width;
   protected int height;
   public void setWidth(int  w){
      width = w;
      height = w;
   }

   public void setHeight(int h){
      width = h;
      height = h;
   }
    public int getWidth(){return width;}
    public int getHeight(){return height;}
   public int getArea(){return width * height;}
}
```

Why the refactored code is respecting LSP

The common behavior of setting width and height of the square and rectangle classes is generalized in the shape class. Both square and rectangle class now extend the shape class. Each class now defines it's own behavior of setting width and height and the behaviors are no more contradicting.

Deliverables

Compile a single word document by filling in the solution part and submit this Word file on LMS. This lab grading policy is as follows: The lab is graded between 0 to 10 marks. The submitted solution can get a maximum of 5 marks. At the end of each lab or in the next lab, there will be a viva related to the tasks. The viva has a weightage of 5 marks. Insert the solution/answer in this document. You must show the implementation of the tasks in the designing tool, along with your completed Word document to get your work graded. You must also submit this Word document on the LMS. In case of any problems with submissions on LMS, submit your Lab assignments by emailing it to nadeem.nawaz@seecs.edu.pk.